

Advanced Database Management Systems (CE-523)

Tutor: Irfanullah Khan

College: Govt Khushal Khan Khattak College Akora Khattak

Lecture: Introduction to Advanced Data Models

What Are Advanced Data Models?

In basic databases (like the relational model), we store data in tables with rows and columns. **Advanced data models** go beyond this to handle more complex real-world data: things like objects, inheritance, and mixed data types. Two important advanced models are **object-relational** and **object-oriented** databases.

1. Object-Relational Model (ORDBMS)

In simple words:

It is an extension of the **relational model**. You still have tables and SQL, but you can also store **complex types** inside a column (e.g., a whole address, a list of phone numbers, or even another small "table" inside a cell).

Key ideas:

- **Same as before:** Tables, rows, columns, and SQL (SELECT, INSERT, UPDATE, DELETE).
- **New:** Columns can hold **user-defined types** (e.g., `ADDRESS`, `PHONE_LIST`) instead of only numbers and text.
- **New:** You can write **functions and procedures** in the database that work on these complex types.

Why it is useful:

When one "thing" (e.g., a customer) has many related pieces (name, address, list of orders), you can group them into one logical type instead of spreading them over many tables. Queries can still use SQL, so it feels familiar.

Example - PostgreSQL (Object-Relational) - Full Walkthrough:

Run all these steps inside `psql` or pgAdmin after connecting to your database.

Step 1 - Create a simple composite type for address

```
CREATE TYPE address_type AS (  
    street  VARCHAR(100),  
    city    VARCHAR(50),  
    zip_code VARCHAR(10)  
);
```

This defines a new type called `address_type`. It is not a table; it is a reusable structure (like a small record) that we can use as a column type in any table.

Step 2 - Create another type for contact info (uses `address_type` inside it)

```
CREATE TYPE contact_type AS (  
    phone          VARCHAR(15),  
    email          VARCHAR(100),  
    home_address   address_type  
);
```

Notice that `contact_type` contains `home_address` which is itself of type `address_type`. This is called a **nested type** - one type inside another.

Step 3 - Create the students table using these types

```
CREATE TABLE students (  
    id          SERIAL PRIMARY KEY,  
    name        VARCHAR(100),  
    roll_no     VARCHAR(20) UNIQUE,  
    contact     contact_type  
);
```

The `contact` column now holds phone, email, AND the full address - all in one column.

Step 4 - Insert records

```
INSERT INTO students (name, roll_no, contact)  
VALUES (  
    'Ali Khan',  
    'CE-001',  
    ROW(  
        '0300-1234567',  
        'ali@example.com',  
        ROW('Main Road', 'Akora Khattak', '24350')  
    )  
);  
  
INSERT INTO students (name, roll_no, contact)  
VALUES (  
    'Sara Ahmed',  
    'CE-002',  
    ROW(  
        '0311-9876543',  
        'sara@example.com',  
        ROW('College Street', 'Nowshera', '24100')  
    )  
);
```

`ROW(...)` builds the composite value. The inner `ROW(...)` is the nested `address_type`.

Step 5 - Select all data

```
SELECT * FROM students;
```

Step 6 - Access a specific field inside the type

```
-- Get just the city from the nested address
SELECT
    name,
    (contact).phone           AS phone,
    (contact).email           AS email,
    ((contact).home_address).city AS city,
    ((contact).home_address).zip_code AS zip
FROM students;
```

Expected output:

name	phone	email	city	zip
Ali Khan	0300-1234567	ali@example.com	Akora Khattak	24350
Sara Ahmed	0311-9876543	sara@example.com	Nowshera	24100

Step 7 - Filter (WHERE) using a field inside the type

```
-- Find students who live in Nowshera
SELECT name, ((contact).home_address).city AS city
FROM students
WHERE ((contact).home_address).city = 'Nowshera';
```

Step 8 - Update a field inside the type

```
-- Update Ali's phone number
-- We must rebuild the whole contact value when updating
UPDATE students
SET contact = ROW(
    '0300-9999999',
    (contact).email,
    (contact).home_address
)
WHERE roll_no = 'CE-001';
```

We keep the existing email and address using `(contact).email` and `(contact).home_address`, and only change the phone.

Step 9 - Clean up (drop table and types)

```
-- Always drop the table first before dropping the types it uses
DROP TABLE students;

DROP TYPE contact_type;
DROP TYPE address_type;
```

Summary of what we used:

Command	Purpose
CREATE TYPE name AS (...)	Define a new composite type
ROW(val1, val2, ...)	Build a composite value to insert
(column).field	Access one field inside a composite column
((column).inner).field	Access a field inside a nested composite
DROP TYPE name	Remove a type (drop table first)

Instead of three separate columns (street, city, zip), we have one address_type column. SQL still works normally; we just access inner fields using (column).field notation.

Additional Object-Relational Examples

These examples extend the concepts above with array types, user-defined functions on types, and domain types — all features of an ORDBMS.

Example A: Array Type (List of Phone Numbers)

In PostgreSQL you can store a list of values directly inside a column using an array type. No separate "phones" table is needed.

Step 1 - Create a table with an array column

```
CREATE TABLE teachers (
  id          SERIAL PRIMARY KEY,
  name        VARCHAR(100),
  phones      VARCHAR(15)[]      -- array of phone numbers
);
```

The [] after a type means "array of that type".

Step 2 - Insert records with multiple phones

```
INSERT INTO teachers (name, phones)
VALUES
  ('Mr. Irfan',   ARRAY['0300-1111111', '0321-2222222']),
  ('Ms. Ayesha',  ARRAY['0333-3333333']),
  ('Mr. Tariq',   ARRAY['0345-4444444', '0312-5555555', '0300-6666666']);
```

`ARRAY[...]` is the literal syntax to write an array value.

Step 3 - Select and read the array

```
-- Show all teachers with their phone lists
SELECT name, phones FROM teachers;
```

Expected output:

name	phones
Mr. Irfan	{0300-1111111,0321-2222222}
Ms. Ayesha	{0333-3333333}
Mr. Tariq	{0345-4444444,0312-5555555,0300-6666666}

Step 4 - Access a specific element (arrays are 1-indexed in PostgreSQL)

```
-- Get only the first phone number of each teacher
SELECT name, phones[1] AS primary_phone FROM teachers;
```

Step 5 - Search: find teachers who have a specific phone

```
-- The @> operator checks if an array contains a value
SELECT name FROM teachers
WHERE phones @> ARRAY['0300-1111111'];
```

Step 6 - Append a new phone to an existing record

```
UPDATE teachers
SET phones = phones || ARRAY['0300-9999999']
WHERE name = 'Ms. Ayesha';
```

`||` is the array concatenation operator.

Step 7 - Count elements in an array

```
SELECT name, array_length(phones, 1) AS num_phones
FROM teachers;
```

Step 8 - Clean up

```
DROP TABLE teachers;
```

Example B: User-Defined Function (Method) on a Type

In ORDBMS you can define **functions** that operate on your custom types, just like methods in OOP. Here we create a function to format the address neatly.

We reuse `address_type` from the earlier example. Run those CREATE TYPE statements first if you dropped them.

```
-- Recreate address_type if needed
CREATE TYPE address_type AS (
    street  VARCHAR(100),
    city    VARCHAR(50),
    zip_code VARCHAR(10)
);
```

Create a function that takes address_type and returns a formatted string

```
CREATE OR REPLACE FUNCTION format_address(addr address_type)
RETURNS TEXT AS $$
BEGIN
    RETURN addr.street || ', ' || addr.city || ' - ' || addr.zip_code;
END;
$$ LANGUAGE plpgsql;
```

`$$` marks the start and end of the function body. `plpgsql` is PostgreSQL's procedural language.

Create a table and insert data

```
CREATE TABLE offices (
    id          SERIAL PRIMARY KEY,
    dept        VARCHAR(100),
    location    address_type
);

INSERT INTO offices (dept, location)
VALUES
    ('Computer Science', ROW('Block A, Main Road', 'Akora Khattak', '24350')),
    ('Physics',          ROW('Block B, Campus Road', 'Nowshera', '24100'));
```

Call the function in a SELECT query

```
SELECT
    dept,
    format_address(location) AS full_address
FROM offices;
```

Expected output:

dept	full_address
Computer Science	Block A, Main Road, Akora Khattak - 24350
Physics	Block B, Campus Road, Nowshera - 24100

The function `format_address()` behaves like a **method** on the `address_type` object — this is the ORDBMS approach to adding behaviour to data.

Clean up

```
DROP TABLE offices;
DROP FUNCTION format_address(address_type);
DROP TYPE address_type;
```

Example C: Domain Type (Constrained Type)

A **domain** is a user-defined type built on an existing base type with extra **constraints** (rules). It is useful to avoid repeating `CHECK` constraints on every table.

```
-- Create a domain: a phone number must match Pakistani format
CREATE DOMAIN pak_phone AS VARCHAR(15)
    CHECK (VALUE ~ '^\\d{4}-\\d{7}$');
```

`~` is PostgreSQL's regex match operator. The pattern `^\\d{4}-\\d{7}$` means: exactly 4 digits, a dash, then 7 digits (e.g., `0300-1234567`).

Use the domain as a column type

```
CREATE TABLE staff (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100),
    phone pak_phone -- automatically validated
);
```

Insert a valid record

```
INSERT INTO staff (name, phone) VALUES ('Ali Khan', '0300-1234567');  
-- Works fine
```

Try to insert an invalid phone (will be rejected)

```
INSERT INTO staff (name, phone) VALUES ('Bad Data', '123');  
-- ERROR: value for domain pak_phone violates check constraint
```

The database itself enforces the format rule — no need for application-level checks.

Clean up

```
DROP TABLE staff;  
DROP DOMAIN pak_phone;
```

2. Object-Oriented Model (OODBMS)

In simple words:

Here the database stores **objects** (like in programming languages such as Java or C++). Each object has:

- **Data** (attributes), and
- **Behaviour** (methods/functions that work on that data).

Key ideas:

- **Objects:** Data is stored as objects, not as rows in tables.
- **Classes:** Objects belong to classes (like "Student", "Course").
- **Inheritance:** A class can inherit from another (e.g., "PartTimeStudent" extends "Student"), so common data and behaviour are reused.
- **Identity:** Each object has a unique ID, so the same object can be referred to from many places without copying it.

Why it is useful:

When your application is written in an object-oriented language, storing objects directly in the database can avoid the extra step of "breaking" objects into rows and "rebuilding" them when reading. It fits well with OOP concepts (encapsulation, inheritance, polymorphism).

Example - Object-Oriented (Conceptual / Pseudo-code):

In an OODBMS we define **classes** and the database stores objects of those classes directly.

```

CLASS Person {
  ATTRIBUTES:
    name : String
    age  : Integer
  METHODS:
    getFullInfo() : String
}

CLASS Student EXTENDS Person {
  ATTRIBUTES:
    roll_no      : String
    enrollment_date : Date
  METHODS:
    enrollInCourse(course : Course)
    getGPA() : Float
}

CLASS PartTimeStudent EXTENDS Student {
  ATTRIBUTES:
    work_hours : Integer
  METHODS:
    getWorkSchedule() : String
}

```

Storing an object:

```

s1 = new Student()
s1.name          = "Ali Khan"
s1.roll_no       = "CE-001"
s1.enrollment_date = 2024-01-15
database.store(s1)

```

Querying with OQL (Object Query Language):

```

-- Get name and GPA of a specific student
SELECT s.name, s.getGPA()
FROM   students s
WHERE  s.roll_no = 'CE-001'

-- Get all PartTimeStudents working more than 20 hours
SELECT p.name, p.work_hours
FROM   students p
WHERE  p IS PartTimeStudent AND p.work_hours > 20

```

Key point: `PartTimeStudent` automatically has `name`, `roll_no`, and `getGPA()` from its parent classes because of **inheritance**. We never copy those attributes; we just extend them.

Practical Object-Oriented Example — PostgreSQL Table Inheritance

PostgreSQL supports **table inheritance**: a child table automatically has all columns of its parent table, just like a subclass inherits attributes from a superclass. This directly implements the OODBMS class hierarchy concept inside a relational/object-relational engine.

We will model the same hierarchy from the pseudo-code above:

Person → Student → PartTimeStudent

Step 1 - Create the base (parent) table: Person

```
CREATE TABLE person (  
  id    SERIAL PRIMARY KEY,  
  name  VARCHAR(100) NOT NULL,  
  age   INTEGER  
);
```

This is the root "class". Every `person` row has `id`, `name`, and `age`.

Step 2 - Create a child table: Student INHERITS Person

```
CREATE TABLE student (  
  roll_no    VARCHAR(20) UNIQUE,  
  enrollment_date DATE  
) INHERITS (person);
```

`INHERITS (person)` means `student` automatically has `id`, `name`, and `age` **plus** its own `roll_no` and `enrollment_date`. We do not repeat the parent columns.

Step 3 - Create a grandchild table: PartTimeStudent INHERITS Student

```
CREATE TABLE part_time_student (  
  work_hours INTEGER  
) INHERITS (student);
```

`part_time_student` now has all columns from both `person` and `student`, plus its own `work_hours`:

- `id`, `name`, `age` (from `person`)
 - `roll_no`, `enrollment_date` (from `student`)
 - `work_hours` (its own)
-

Step 4 - Insert rows into each table

```

-- A plain person (not a student)
INSERT INTO person (name, age)
VALUES ('Dr. Ahmad', 45);

-- A regular student (also appears in person queries)
INSERT INTO student (name, age, roll_no, enrollment_date)
VALUES ('Ali Khan', 20, 'CE-001', '2024-01-15');

INSERT INTO student (name, age, roll_no, enrollment_date)
VALUES ('Sara Ahmed', 21, 'CE-002', '2024-01-15');

-- A part-time student (also appears in student and person queries)
INSERT INTO part_time_student (name, age, roll_no, enrollment_date, work_hours)
VALUES ('Bilal Saeed', 22, 'CE-003', '2024-01-15', 25);

```

Step 5 - Query the parent table (polymorphism)

```

-- This returns ALL persons: Dr. Ahmad, Ali, Sara, AND Bilal
-- because student and part_time_student are subclasses of person
SELECT name, age FROM person;

```

Expected output:

name	age
Dr. Ahmad	45
Ali Khan	20
Sara Ahmed	21
Bilal Saeed	22

This is **polymorphism**: querying the parent returns rows from all child tables too.

Step 6 - Query only direct rows of person (exclude children)

```

-- ONLY keyword restricts to rows stored directly in 'person', not children
SELECT name, age FROM ONLY person;

```

Expected output:

name	age
Dr. Ahmad	45

Step 7 - Query the student table

```
-- Returns Ali, Sara, AND Bilal (because part_time_student inherits student)
SELECT name, roll_no, enrollment_date FROM student;
```

Expected output:

name	roll_no	enrollment_date
Ali Khan	CE-001	2024-01-15
Sara Ahmed	CE-002	2024-01-15
Bilal Saeed	CE-003	2024-01-15

Step 8 - Query only part-time students with their work hours

```
SELECT name, roll_no, work_hours FROM part_time_student;
```

Expected output:

name	roll_no	work_hours
Bilal Saeed	CE-003	25

Step 9 - Add a function (method) on the student type

We simulate the `getGPA()` method by creating a function that takes a student's roll number and returns a GPA (for demonstration, we calculate it from a grades table).

```

-- Simple grades table to support the GPA function
CREATE TABLE grades (
    roll_no  VARCHAR(20),
    subject  VARCHAR(50),
    marks    INTEGER
);

INSERT INTO grades VALUES
    ('CE-001', 'Database', 85),
    ('CE-001', 'Networks', 78),
    ('CE-002', 'Database', 90),
    ('CE-002', 'Networks', 88),
    ('CE-003', 'Database', 72),
    ('CE-003', 'Networks', 65);

-- The "method": function getGPA(roll_no)
CREATE OR REPLACE FUNCTION get_gpa(p_roll_no VARCHAR)
RETURNS NUMERIC AS $$
DECLARE
    avg_marks NUMERIC;
BEGIN
    SELECT AVG(marks) INTO avg_marks
    FROM grades
    WHERE roll_no = p_roll_no;

    -- Simple scale: avg/10 rounded to 2 decimal places
    RETURN ROUND(avg_marks / 10.0, 2);
END;
$$ LANGUAGE plpgsql;

```

Step 10 - Call the function in a query (OQL-style)

```

-- Equivalent of: SELECT s.name, s.getGPA() FROM students s
SELECT
    name,
    roll_no,
    get_gpa(roll_no) AS gpa
FROM student;

```

Expected output:

name	roll_no	gpa
Ali Khan	CE-001	8.15
Sara Ahmed	CE-002	8.90
Bilal Saeed	CE-003	6.85

Step 11 - Filter part-time students working over 20 hours (OQL equivalent)

```
-- OQL: SELECT p.name, p.work_hours FROM students p
--      WHERE p IS PartTimeStudent AND p.work_hours > 20
SELECT name, roll_no, work_hours
FROM part_time_student
WHERE work_hours > 20;
```

Expected output:

name	roll_no	work_hours
Bilal Saeed	CE-003	25

Step 12 - Check which table a row actually lives in

```
-- tableoid::regclass shows the actual physical table of each row
SELECT name, age, tableoid::regclass AS stored_in
FROM person;
```

Expected output:

name	age	stored_in
Dr. Ahmad	45	person
Ali Khan	20	student
Sara Ahmed	21	student
Bilal Saeed	22	part_time_student

This clearly shows how each object is stored in its own table while still being reachable through the parent.

Step 13 - Clean up (drop children before parent)

```
-- Drop in reverse order (child before parent)
DROP TABLE part_time_student;
DROP TABLE student;
DROP TABLE person;
DROP TABLE grades;
DROP FUNCTION get_gpa(VARCHAR);
```

Summary: OODBMS concepts mapped to PostgreSQL features

OODBMS concept	PostgreSQL feature
Class	Table
Subclass	<code>INHERITS</code> (child table)
Object instance	Row
Object identity	<code>id</code> (primary key) / <code>tableoid</code>
Method	Function (<code>CREATE FUNCTION</code>)
Inheritance	<code>INHERITS</code> — child has all parent columns
Polymorphism	Querying parent returns rows from all children
ONLY keyword	Restricts query to direct rows only

Quick Comparison

Feature	Relational (RDBMS)	Object-Relational (ORDBMS)	Object-Oriented (OODBMS)
Basic unit	Table, row, column	Table + complex types	Object, class
Query language	SQL	Extended SQL	OQL or language APIs
Complex data	Many tables, joins	User-defined types in columns	Nested objects
Inheritance	No	Limited / via types	Yes (class hierarchy)

Summary

- **Object-relational:** “Relational database + ability to define your own data types and store richer structures in columns.” Still SQL-based.
- **Object-oriented:** “Database that stores objects and classes, with inheritance and methods.” Closer to how OOP languages work.

PostgreSQL is a well-known example of an **object-relational** database: it supports tables and SQL plus user-defined types, arrays, and JSON, so it is a good system to learn these concepts.

PostgreSQL Installation

On Windows

1. Download PostgreSQL

- Go to: <https://www.postgresql.org/download/windows/>
- Click “Download the installer” (from EDB).
- Download the latest stable version (e.g., PostgreSQL 16 or 17).

2. Run the installer

- Double-click the downloaded `.exe` file.
- Choose installation directory (default is fine).

- Select components: **PostgreSQL Server**, **pgAdmin 4**, **Stack Builder** (optional), and **Command Line Tools** (recommended).

3. Set password for superuser

- When asked, set a password for the `postgres` user. Remember it; you will need it to connect.

4. Port

- Default port is `5432`. Keep it unless you have a conflict.

5. Finish

- Complete the setup. You can launch **Stack Builder** at the end if you need extra tools; otherwise click **Finish**.

6. Check installation

- Open **Command Prompt** or **PowerShell** and run:

```
psql --version
```

- If you see a version number, PostgreSQL is installed.

On Mac

Option A: Official installer

1. Go to: <https://www.postgresql.org/download/macos/>
2. Download the installer (e.g., from EDB or Postgres.app).
3. Run the installer and follow the steps (set password for `postgres`, port 5432).
4. Ensure "Command Line Tools" are installed so that `psql` is available in the terminal.

Option B: Using Homebrew (common on Mac)

1. Install Homebrew if you do not have it: <https://brew.sh>
2. Open **Terminal** and run:

```
brew update  
brew install postgresql@16
```

3. Start the service:

```
brew services start postgresql@16
```

4. (Optional) Add to PATH so `psql` is easy to use:

```
echo 'export PATH="/opt/homebrew/opt/postgresql@16/bin:$PATH"' >> ~/.zshrc  
source ~/.zshrc
```

(Use `~/.bash_profile` if you use Bash instead of Zsh.)

5. Check:

```
psql --version
```

Basic Scripts to Start Using PostgreSQL

1. Connect to the database (command line)

Windows (after installation):

- From **Start**, open "SQL Shell (psql)" or open Command Prompt and run:

```
psql -U postgres
```

- Enter the password you set during installation.

Mac / Linux:

```
psql -U postgres
```

(Enter password when asked.)

To connect to a specific database (after you create one):

```
psql -U postgres -d mydb
```

2. Create a database and a table

Run these in the `psql` prompt (type each line and press Enter):

```
-- Create a database
CREATE DATABASE student_db;

-- Connect to it (in psql)
\c student_db

-- Create a simple table
CREATE TABLE students (
  id          SERIAL PRIMARY KEY,
  name        VARCHAR(100),
  roll_no     VARCHAR(20),
  enrollment_date DATE
);

-- Insert some data
INSERT INTO students (name, roll_no, enrollment_date)
VALUES ('Ali Khan', 'CE-001', '2024-01-15'),
      ('Sara Ahmed', 'CE-002', '2024-01-15');

-- View the data
SELECT * FROM students;
```

3. Useful psql commands (meta-commands)

Command	Description
<code>\l</code>	List all databases
<code>\c dbname</code>	Connect to database <code>dbname</code>
<code>\dt</code>	List tables in current DB
<code>\d table_name</code>	Describe a table
<code>\q</code>	Quit psql
<code>\?</code>	Help on psql commands
<code>\h</code>	Help on SQL commands

4. Basic SQL script file

Save this as `basic_queries.sql` and run it with:

```
psql -U postgres -d student_db -f basic_queries.sql
```

```
-- basic_queries.sql
-- Make sure you are in the right database: student_db

-- Select all students
SELECT * FROM students;

-- Select by condition
SELECT name, roll_no FROM students WHERE roll_no = 'CE-001';

-- Update a record
UPDATE students SET name = 'Ali Khan Khattak' WHERE roll_no = 'CE-001';

-- Count rows
SELECT COUNT(*) FROM students;

-- Delete a record (use carefully!)
-- DELETE FROM students WHERE roll_no = 'CE-002';
```

5. Quick reference: start and stop (Mac with Homebrew)

```
# Start PostgreSQL
brew services start postgresql@16

# Stop PostgreSQL
brew services stop postgresql@16

# Check status
brew services list
```

On **Windows**, PostgreSQL usually runs as a service and starts automatically; you can start/stop it from **Services** (services.msc) or from the Start menu if the installer added shortcuts.

End of Lecture 1.

Practice creating a database, a table, and running the basic scripts above to get comfortable with PostgreSQL.

Lecture 2: File Organization Concepts

Why Do We Care About File Organization?

A database stores data in **files** on disk. How we **organize** those files - where we put records and how we arrange them - affects:

- **How fast** we can find a record (search),
- **How fast** we can add or delete a record (insert/delete),
- **How much space** we use and how we use disk (efficiency).

File organization means the way we **physically arrange and store records** in a file so that we can access them efficiently.

Basic Terms

Term	Meaning
File	A collection of related records stored on disk.
Record	One row of data (e.g., one student, one transaction).
Field	One piece of data inside a record (e.g., name, id, date).
Block (or Page)	The unit that the OS reads/writes from disk. Many records may fit in one block.
Access method	How we read data: sequential (one after another) or direct (jump to a specific record).

Types of File Organization

1. Heap (Pile) File Organization

In simple words: Records are stored in **no particular order**. New records are added at the **end** of the file (or in any free space).

- **Insert:** Very fast - just append.
 - **Search:** Slow - we may have to scan the whole file.
 - **Use when:** We add data often and search rarely, or we often scan the whole file.
-

2. Sequential File Organization

In simple words: Records are stored in **sorted order** based on a **key field** (e.g., roll number, id).

- **Insert/Delete:** Slower - we must keep the order, so we may need to shift or rewrite.
 - **Search:** Faster if we use the sort key - we can use binary search or stop early.
 - **Scan in order:** Very efficient (e.g., reports sorted by id).
 - **Use when:** We often process data in order (e.g., batch reports).
-

3. Indexed File Organization

In simple words: We keep the main data file and add an **index** - a separate structure that tells us **where** to find a record (like a book index: keyword → page number).

- **Index:** Contains (key value, pointer to record). Sorted by key for fast lookup.
- **Search:** Fast - look up the key in the index, then go directly to the record.
- **Insert/Delete:** We must update both the file and the index.
- **Use when:** We need fast access by a specific field (e.g., search by student id).

Types of indexes:

- **Primary index:** On the ordering key of the file (unique, sorted).
- **Secondary index:** On another field; allows fast search by that field too (e.g., search by name).

4. Hashed File Organization

In simple words: We use a **hash function** on the key. The function gives us a **bucket number** (or block number). We store the record in that bucket.

- **Hash function:** e.g., $h(\text{key}) = \text{key} \% N$ - bucket 0 to N-1.
- **Search by key:** Very fast - compute hash, go to that bucket, search only there.
- **Search by non-key:** Not supported by the hash; we need a full scan or another index.
- **Insert:** Fast if the bucket has space.
- **Problem: Collisions** - two keys can hash to the same bucket. We handle them (e.g., chaining, open addressing).

Use when: We mostly access by **exact key** (e.g., get record by id).

5. B-Tree / B+ Tree Organization

In simple words: Data (or pointers to data) is stored in a **tree** structure. The tree is balanced so that we rarely need to read many blocks to find a record.

- **Search:** Fast - follow the tree from root to leaf (logarithmic number of block accesses).
 - **Range queries:** Very good in **B+ trees** - leaves are linked, so "all records where id between 10 and 50" is efficient.
 - **Insert/Delete:** The tree is rebalanced so it stays efficient.
 - **Use when:** We need both fast exact search and range search; this is what most DBMSs use for indexes.
-

Access Methods: Sequential vs Direct

Access method	How it works	Best file type
Sequential	Read records one after another from start to end.	Sequential, heap (for full scan).
Direct (random)	Jump to a specific record using address or key (index/hash).	Indexed, hashed.

Quick Comparison of File Organizations

Organization	Insert	Search by key	Range search	Typical use
Heap	Fast	Slow	Slow	Bulk load, full scan
Sequential	Slow	Medium (binary search)	Fast (in order)	Sorted batch processing
Indexed	Medium	Fast	Depends on index	General purpose, multiple access paths
Hashed	Fast	Very fast (exact key)	Not supported	Exact key lookup
B-Tree / B+ Tree	Medium	Fast	Fast	Default in most DBMSs

Summary for Students

- **File organization** = how we physically store and arrange records in a file.
- **Heap**: no order; fast insert, slow search.
- **Sequential**: sorted by key; good for in-order processing.
- **Indexed**: file + index; fast search by key(s).
- **Hashed**: key → bucket; very fast exact key access; no order.
- **B-Tree/B+ Tree**: balanced tree; fast key and range access; widely used in DBMSs.

Choosing the right organization depends on how we **access** data (by key, in order, full scan) and how we **update** it (many inserts vs few).

End of Lecture 2.